High Level Programming Languages

by
munawwar_ali@yahoo.com

Programming Languages Generations

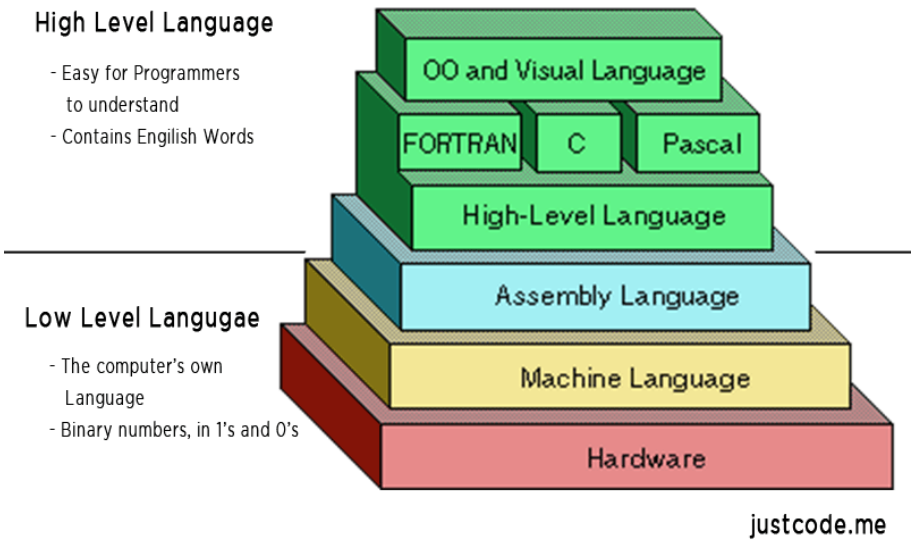
1st Generation (1GL): 1940 to 1950 : Machine Language : 0s and 1s

2nd Generation (2GL): 1950 to 1960 : Assembly Language : Mnemonics (short code words understood by CPU)

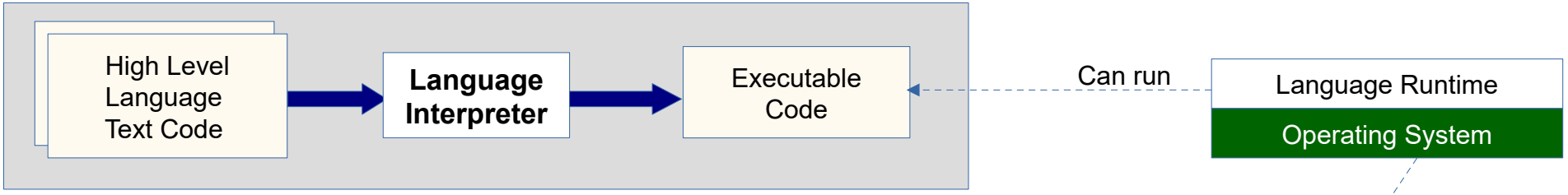
3rd Generation (3GL): 1960 to 1980 : High Level Languages : FORTRAN, COBOL, BASIC, C, C++

4th Generation (4GL): 1980 onward: Very High Level Languages: SQL, Python, Ruby, Java, C#

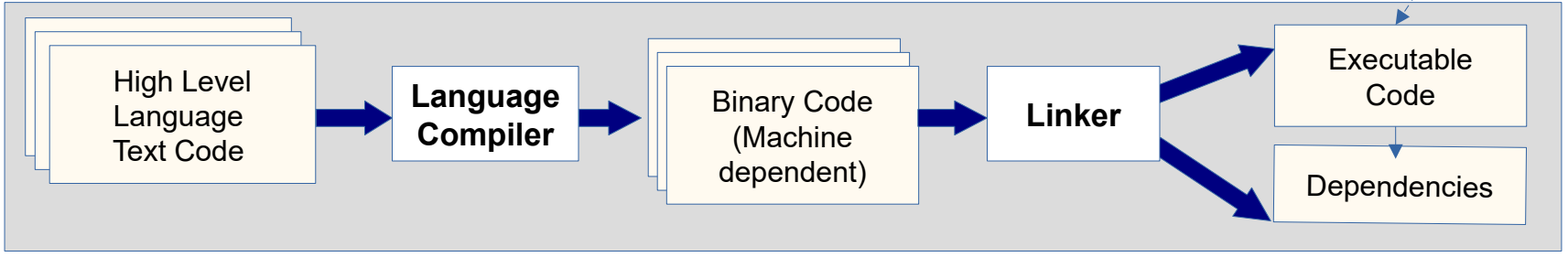
5th Generation (5GL): 1980 onward: Natural Language: AI & Machine Learning



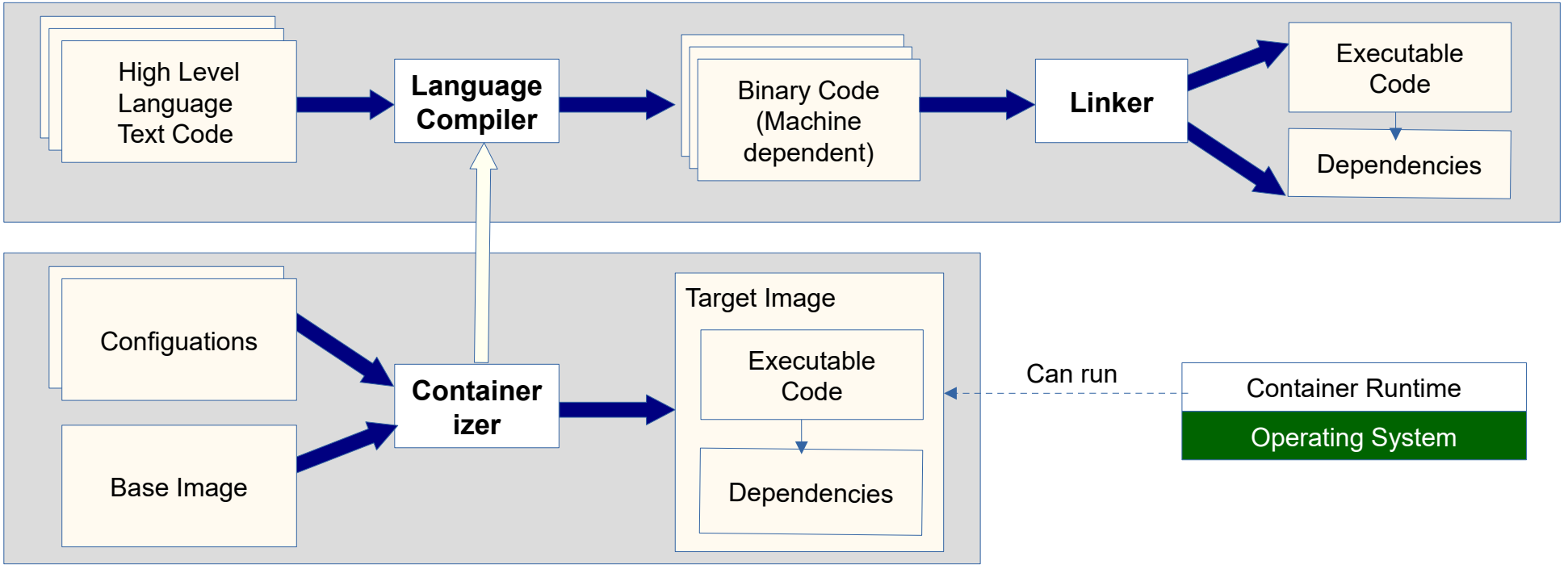
Programming Languages - Interpreters



High Level Programming Languages - Compilers



High Level Programming Languages - Containerizers



Data Types & Variables

A data type defines how the value of an object is stored and how many bytes are available for it.

Value types:
There are the types used to store numerical values such as integer, decimal, etc.

A 16-bit **unsigned integer** type => would have 2 bytes to store the values from 0 to 2^{16}-1
A 16-bit **signed integer** type => would have 2 bytes to store the values 0 to 2^{-15} and -1 to -2^{15}

A 32-bit **unsigned integer** type => would have 4 bytes to store the values from 0 to 2^{32}-1
A 32-bit **signed integer** type => would have 4 bytes to store the values 0 to 2^{-31} and -1 to -2^{31}

A **32-bit floating point** type => would use 4 bytes (See IEEE 754 notation)
A 64-**bit floating point** type => would use 8 bytes (See IEEE 754 notation)

A **8-bit character** type => will store 1 byte of information (ASCII Character)
A **16-bit character** type => will store 2 bytes of information (UNICODE Character)

An **array** type => will hold multiple values for a given data type
An **integer array** type of size 10 => will hold 10 sequential values (each of type integer)
An **character array** type of size 10 => will hold 10 sequential values (each of type character)

A **string** type=> will hold variable sequential values (each of type character) e.g. “Hello”, “Hi”.
* Depending on the language/system, the string type will hold 8-bit or 16-bit values.

Data Encoding Standards

Encoding standards provide an structured way to store and use data in a given context.

ASCII (American Standard Code for Information Interchange):

7-bit character encoding for text representation. e.g. A = 65, a = 97

Unicode (Universal Character Encoding)

Defines character sets for all languages of the world and many symbols and sets

UTF (Unicode Transformational Format): UTF-8 / UTF-16 / UTF-16

Variable length encoding – depending on the value of the character

UTF-8 => 1 to 4 bytes; UTF-16 => 2 or 4 bytes; UTF-32 => 4 bytes

Base64:

Used to represent binary information in text format.

MIME (Multipurpose Internet Mail Extensions)

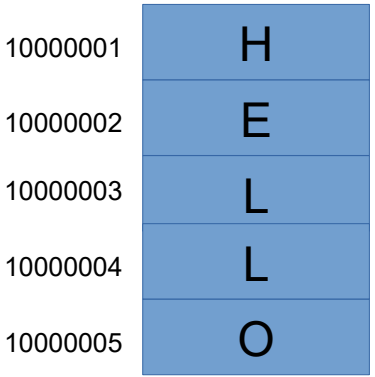
Understood by browsers for data transfer of different file or data formats

e.g. text/plain => text format; audio/mp3 => mp3 file format; etc.

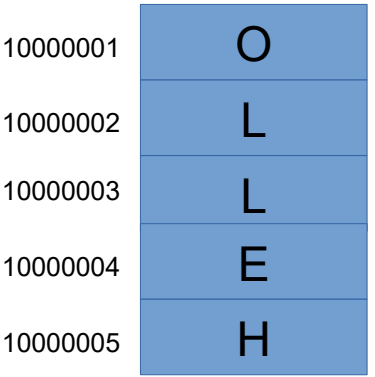
Endianness

The value of a data type can have more than one bytes. The underlying system can decide to store least significant bytes first or most significant bytes first. Due to this, there can be two layout orders :

Big Endian: stores the MSB first
e.g. string “HELLO” =>

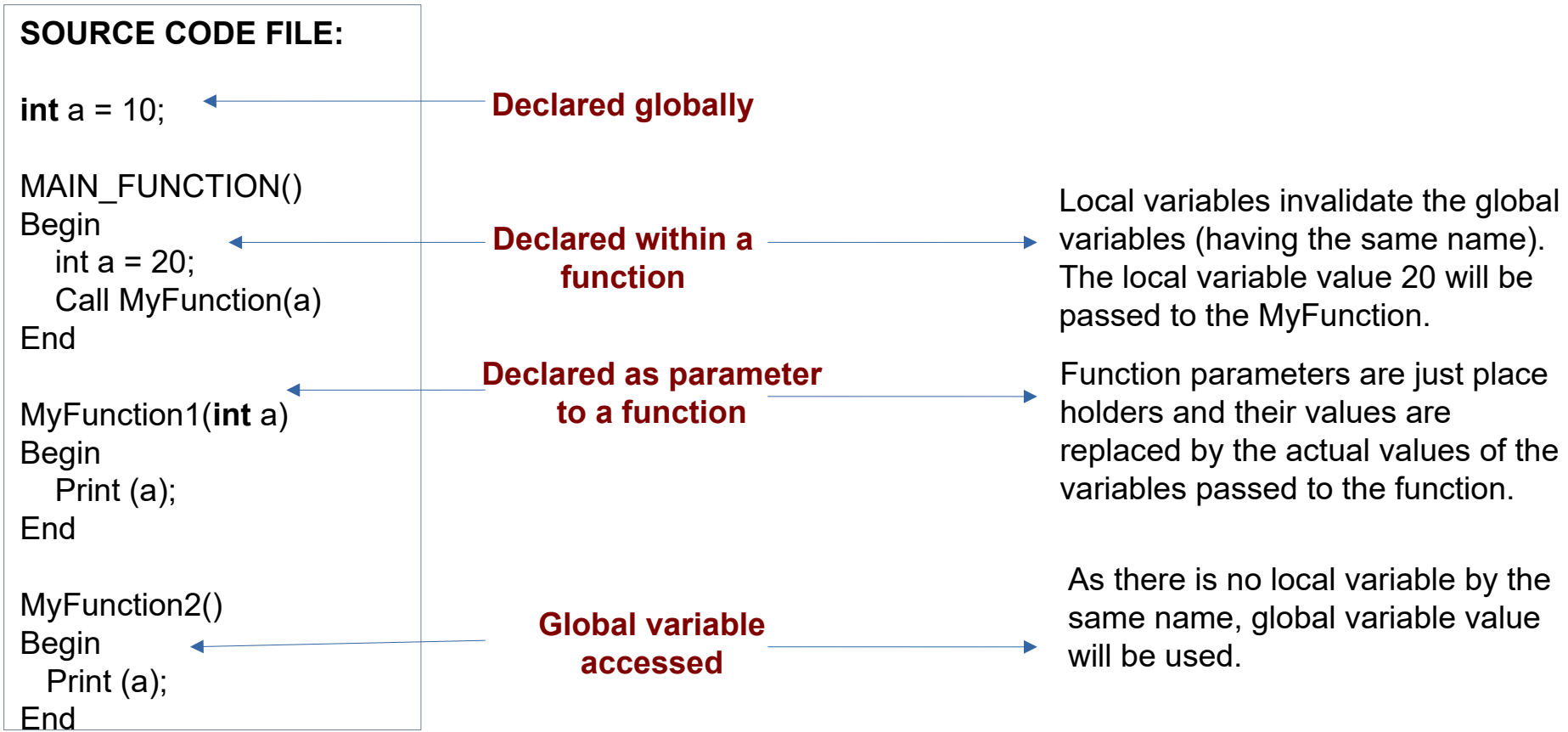


Little Endian: stores the LSB first:
e.g. string “HELLO” =>



Variable Scope

A local (lowest) scope invalidates the global (highest) scope



Assignment & Comparison

The high level programming languages generally use the **= operator** for assignment and for comparison. Some languages use **== operator** to distinguish from assignment to comparison.

Assignment: LHS = RHS

```
int a = 10;    // assign an integer value of 10 to variable a
int b = a;     // assign value of variable a to variable b
b = b + 1      //  calculates the expression on the RHS and assign the calculated value to LHS
b+= 1          // short form of b = b + 1
b++            // short form of b = b + 1
++b            // short form of b = b + 1
```

Comparison: Is LHS == RHS ?

```
int a = 10;                          // assignment
int b = a;                           // assignment
if(a == b)                           // comparison
        print("a is equal to b")
else
        print("a is not equal to b")
```

Loops

The high level programming languages generally support following loop constructs:

For Loop:

```
int my_array[5] = {1,2,3,4,5};
for(index=0;                          // first
    index < sizeof(my_array);         // second
    Index++)                          // third
{ // begin scope
      print(my_array[index]);
} //end scope
```

For loop takes three statements separated by semicolon (C example).

First statement is assignment.
Second is a condition;
Third is an expression.

Ass long as the condition is true, the for loop executes the code within the scope and after every iteration, executes the expression (3rd statement) before checking the condition.

While Loop:

```
int my_array[5] = {1,2,3,4,5};
int index = 0;
while (index < sizeof(my_array))
{// begin scope
      print(my_array[index]);
      index++;
} //end scope
```

While loop takes a condition.

Ass long as the condition is true, it executes the code within the scope.

The code within the scope must update the variables so that condition is eventually false. Otherwise the loop will run indefinitely.

Data Structures

What are data structures?

Data structures store data in a structured fashion using primitive data types supported by a programming language.

Primitive data types: Every programming language supports primitive types that have well known memory size requirements, e,g,:

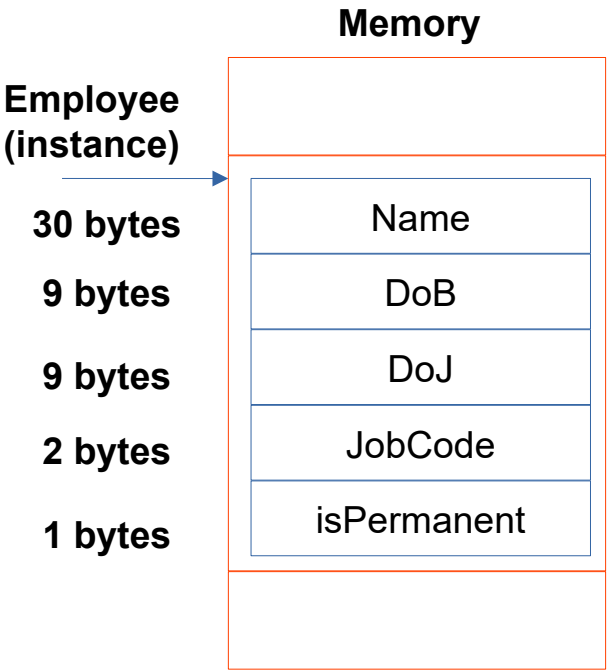
Number types – integer and decimal number types
Char – a single text character
String or char array – literal or text data
Boolean – True or False
Date Time – date and/or time values

Custom data types: custom data types can be created using the primitive types e.g.
Employee Structure:
{

char[30] Name	=> a string literal
DateTime DoB	=> date time value
DateTime DoJ	=> date time value
int JobCode	=> integer
bool isPermanent	=> boolean

}

Total size of the above data structure woulf be sum of individual member size:
30 + 9 + 9 + 2 + 1 = 51 bytes (sequential bytes on memory)

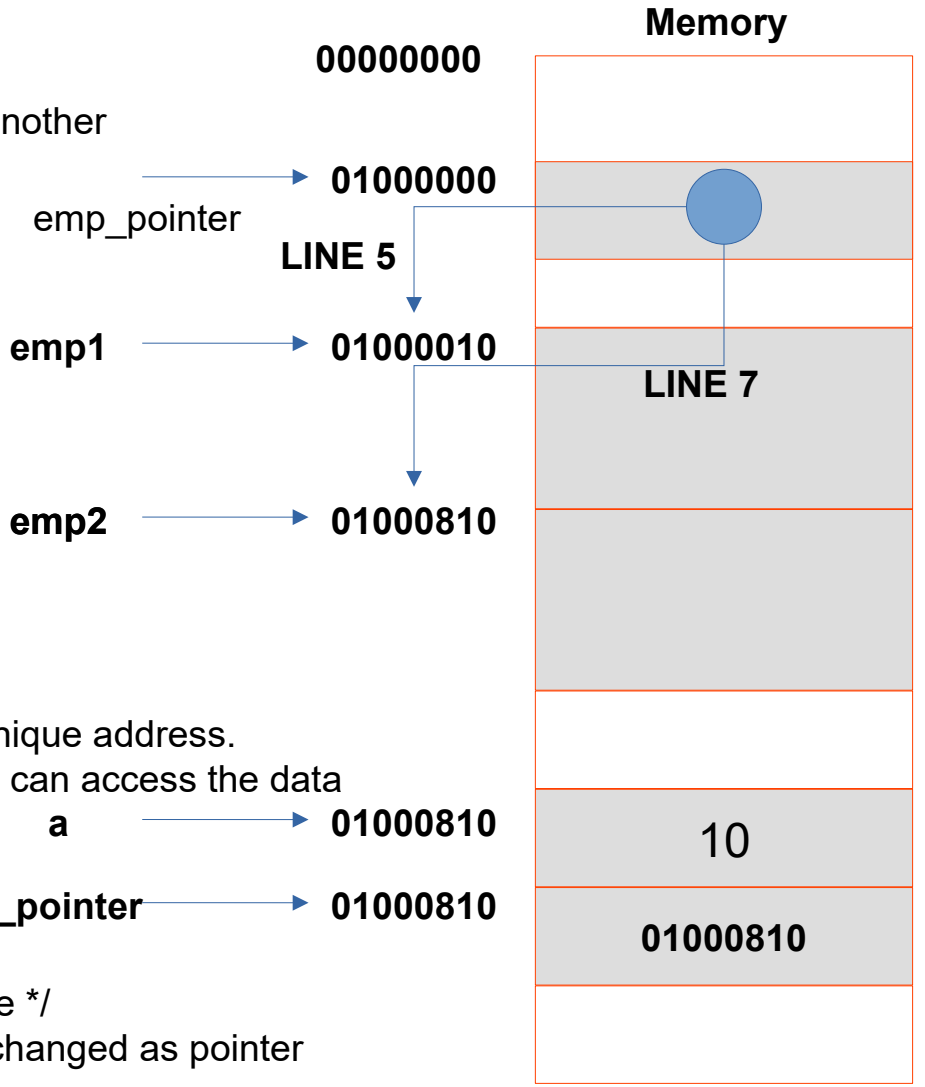


What are data pointers?

A data pointer is a variable that stores the memory address of another variable or data structure.

Following is an example:

```
LINE 1      Employee* emp_pointer = NULL;
LINE 2
LINE 3      Employee emp1, emp2;
LINE 4
LINE 5      emp_pointer = address_of(emp1)
LINE 6
LINE 7      emp_pointer = address_of(emp2)
```



Every primitive type or data structure stored in memory has a unique address.
A pointer that has access to the address of the type or structure can access the data of the structure ot type

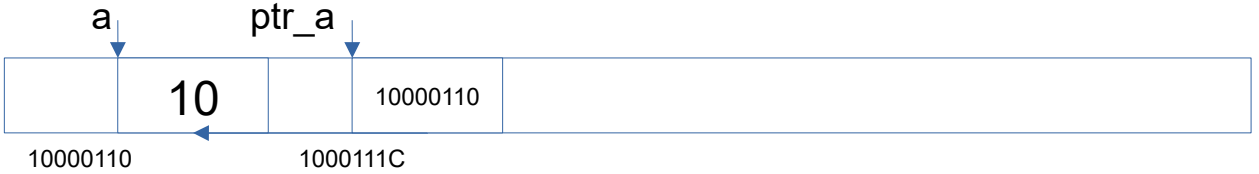
Example:

```
Int a;
Int* a_pointer = &a;  /* & here returns the address of the variable */
*a_pointer = 10                    // value of variable a will be changed as pointer points to it
```

Pointers (Example)

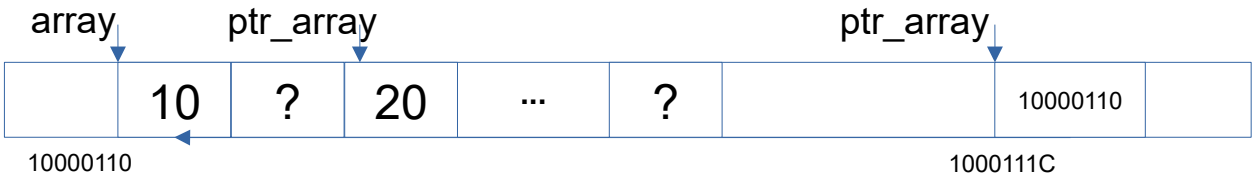
integer pointer

Int a = 10; int* ptr_int = &a;



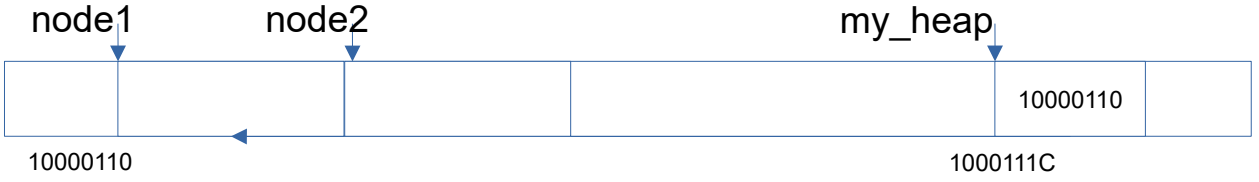
Array pointer

int[10] arr; int** ptr_array = &a;
int** ptr_a = ptr_array + 2;
ptr_a[0] = 20;
ptr_array [0] = 10;



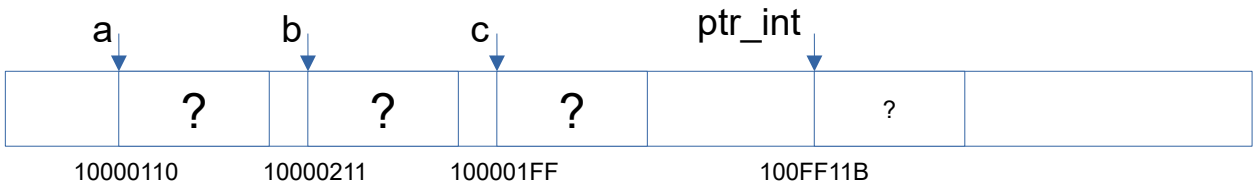
struc pointer

void** myHeap =
malloc(5*sizeof(Node));
struct Node node1 = new Node();
struct Node node2 = new Node();
*myHeap = node1;
myHeap = myHeap + sizeof(Node);
*myheap = node2;



Pointers (Exercise)

```
Int a=10, b=20, c;
int* ptr_int;

ptr_int = &c;
*ptr_int = 50;

ptr_int = &a;
*ptr_int = a + 10;

ptr_int = &b;
ptr_int = b + 10;
```



What is array type?

An array is a data structure that can hold multiple values of same type. The size or length of the array array is fixed.

For example:

int array[5] => create a sequential memory allocation equivalent to 5 integers

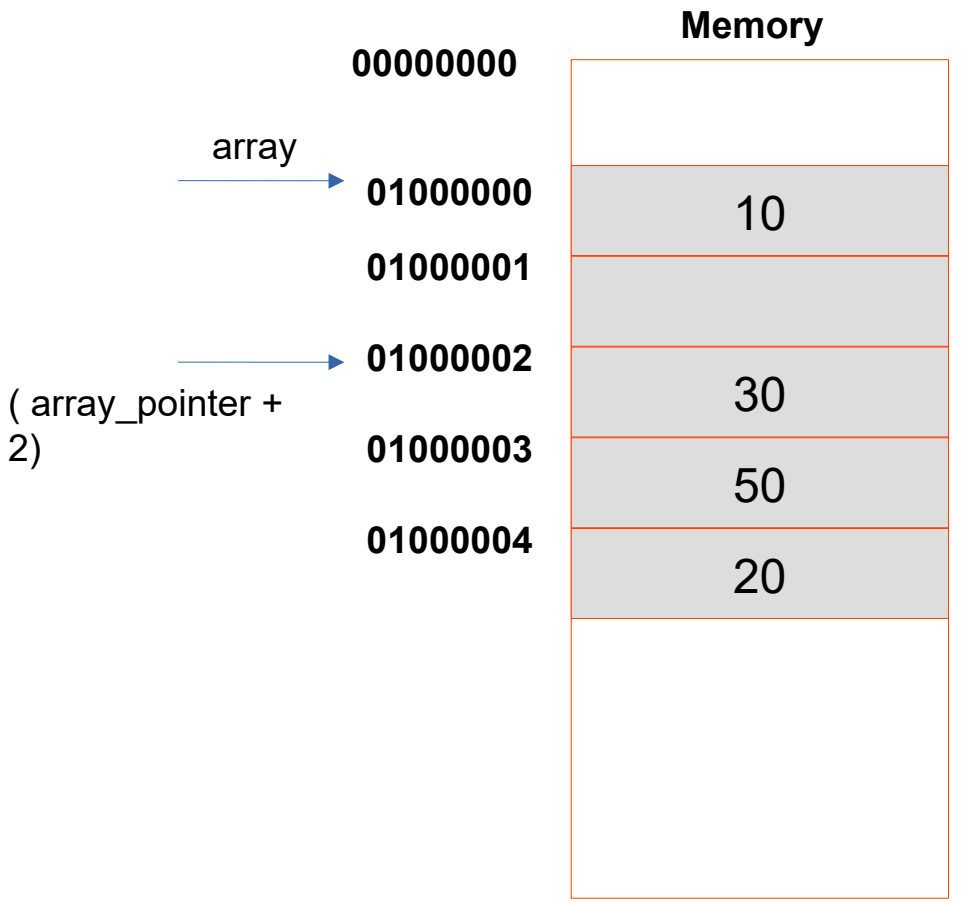
array[0] = 10
array[4] = 20

int * array_pointer = address_of(a);

* (array_pointer + 2) = 30;

* (array_pointer + 3) = 50;

What will happen if:



What is a list type?

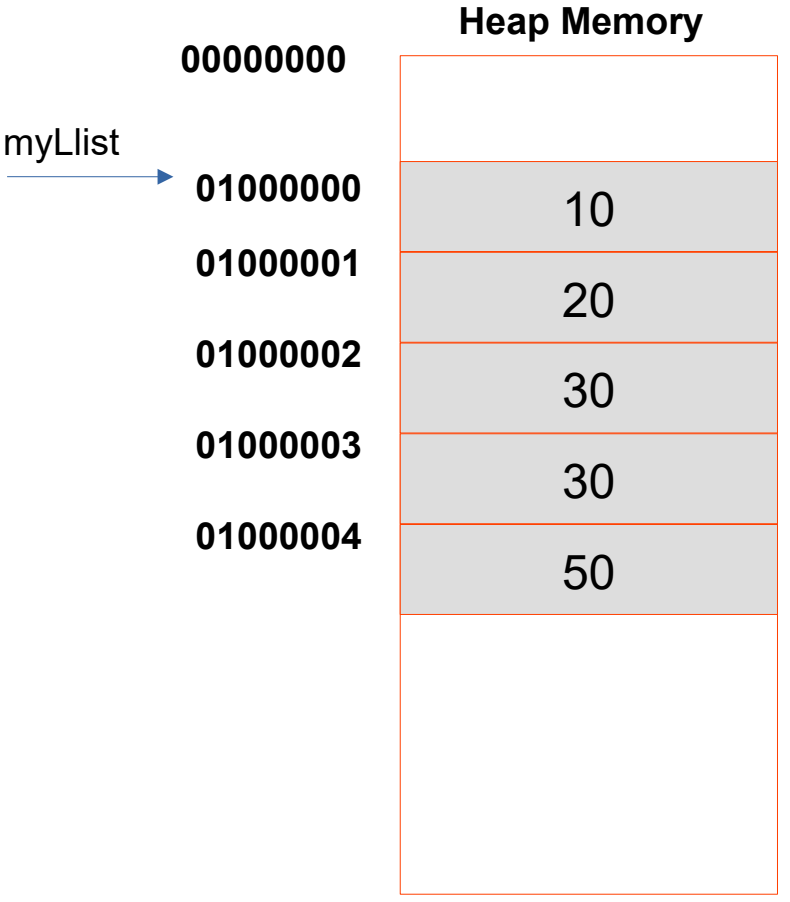
An list is a dynamic array and its size can change based on the number of elements added (or removed) from it.

The list can dynamically change its memory allocation to fit more elements as they are added to it. So the address of the list may keep changing.

The elements of the list are still stored in a continuous memory cells.

For example:

```
List<int> myList = new List<int>;
myList.Add(10);
myList.Add(20);
myList.Add(30);
myList.Add(30);
myList.Add(50);
```



What is a linked list type?

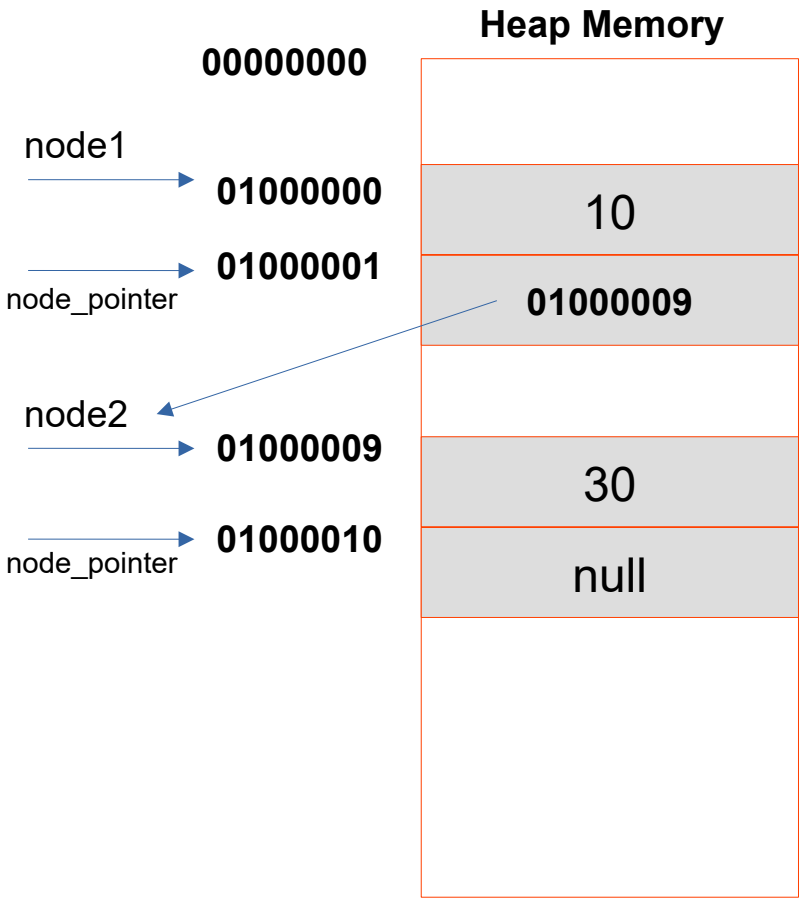
An linked list is a special list array where each member (called Node) of the list points to the next member.

This optimizes the memory allocation of the list as the entire memory need not be reallocated everytime a new member is added.

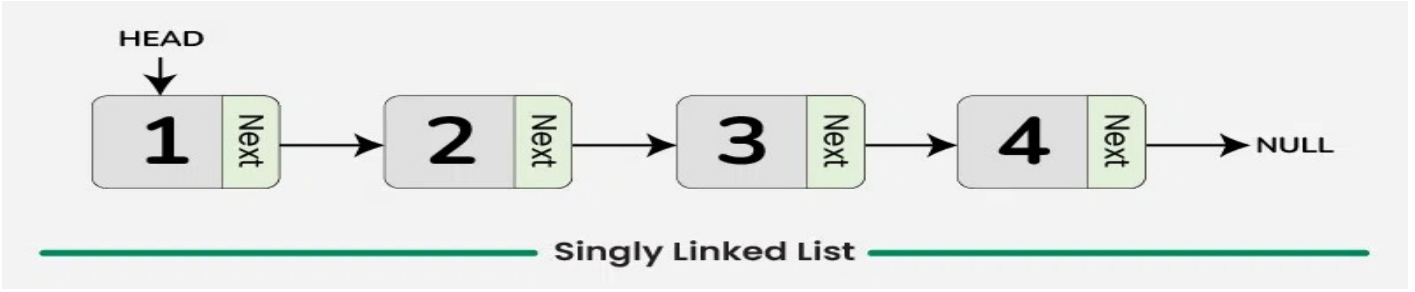
Only the memory required for the new member is allocated and its pointer is set to the previous element.

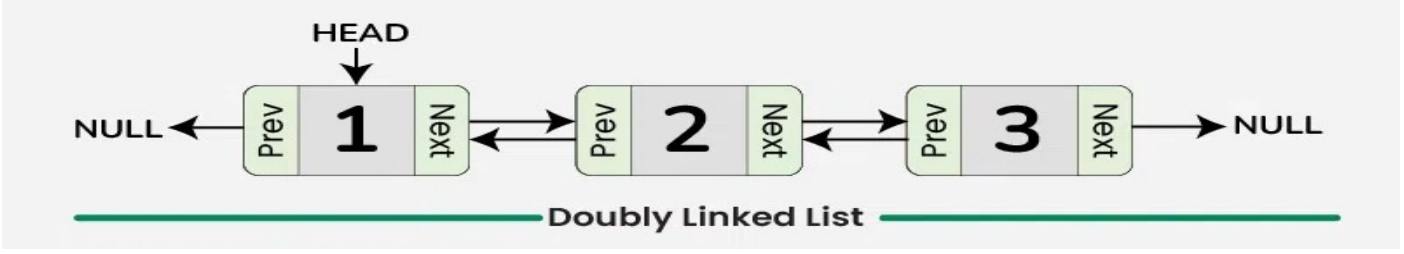
For example:

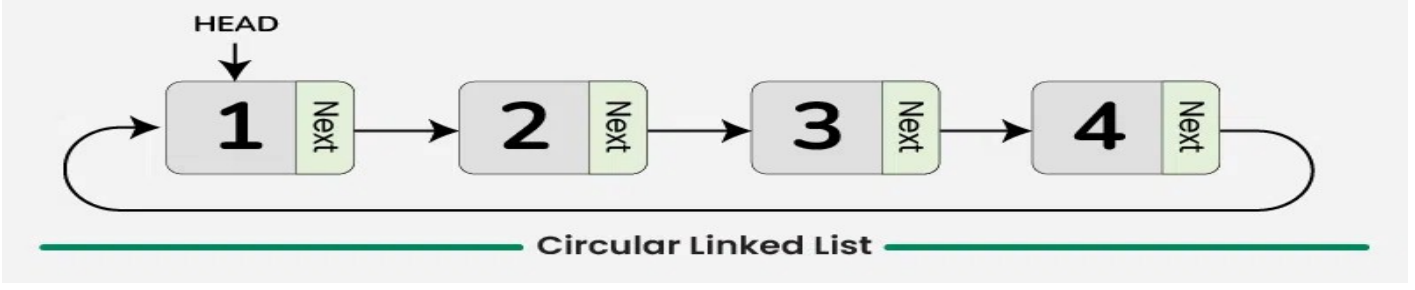
```
Struct Node
{
  int data;
  Struct Node* next_node;
}

List<Node> = myLilnkedList = new List<Node>();
Node node1 = new Node();
Node node2 = new Node();
node1.node_pointer = node2;
MyLilnkedList.Add(node1);
MyLilnkedList.Add(node2)
```



Linked List Types



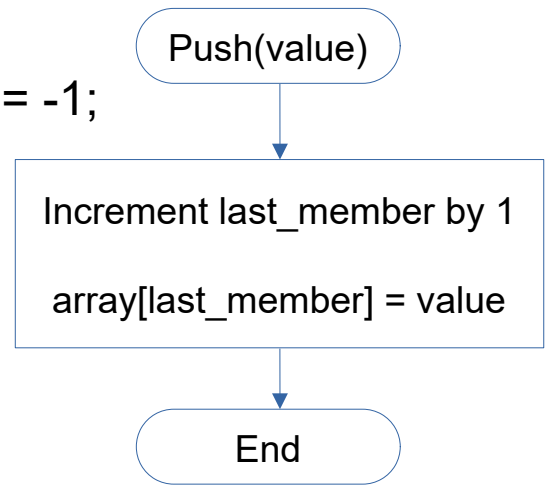


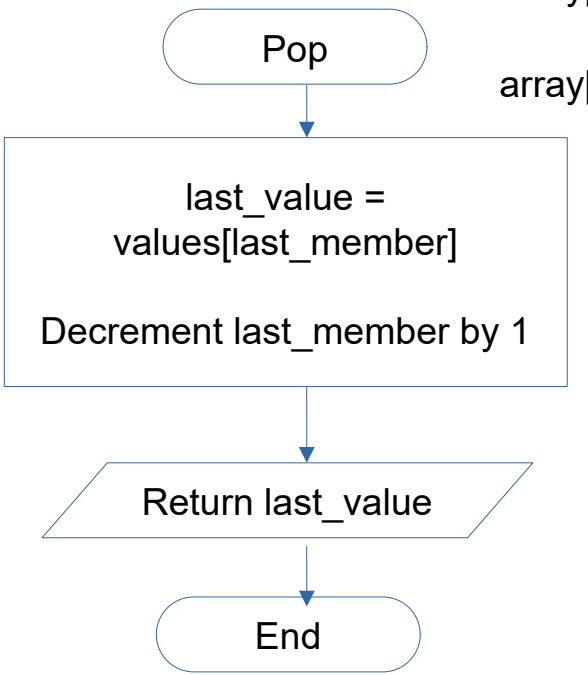


Stack Data Type (LIFO)

- Stack is data structure that can hold multiple items
- An Item can be added to the Stack using **push** method
- An Item can be removed from the Stack using **pop** method
- The Stack has a LIFO (Last In First Out) policy. The **pop** method removes the item that was added at the last
- Stack uses a **pointer** member to track last item added

```
struct Stack
{
 Int [5] array;
 int last_member = -1;
}

Stack s1;
s1.push(10);
s1.push(20);
```

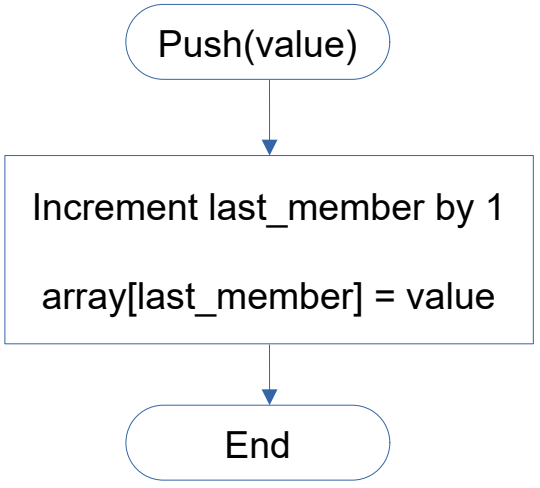


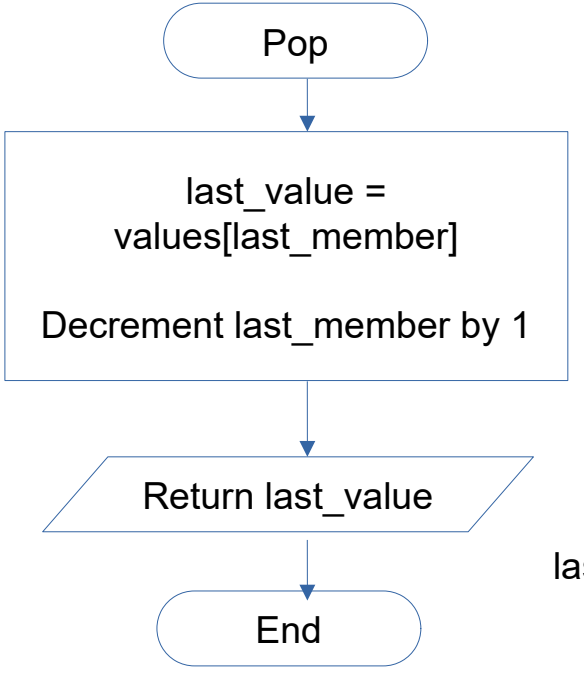


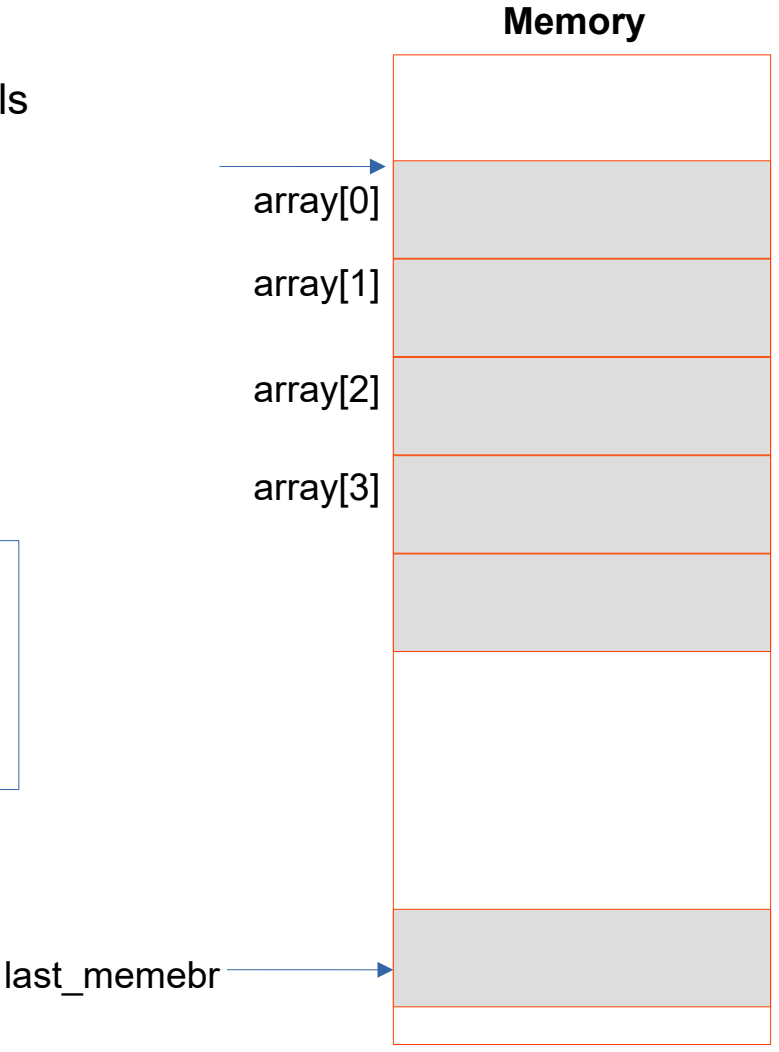
Stack Data Type (Continued)

- Stack s1;
- s1.push(11);
- s1.push(12);
- s1.pop();
- s1.push(13);
- s1.pop();
- s1.push(14);

Update the values of memory cells after execution of the code



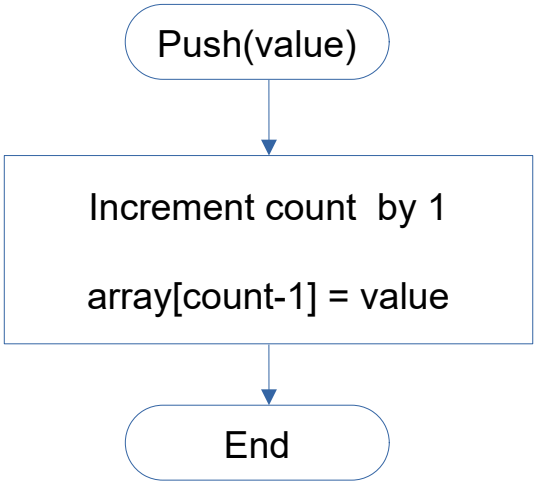


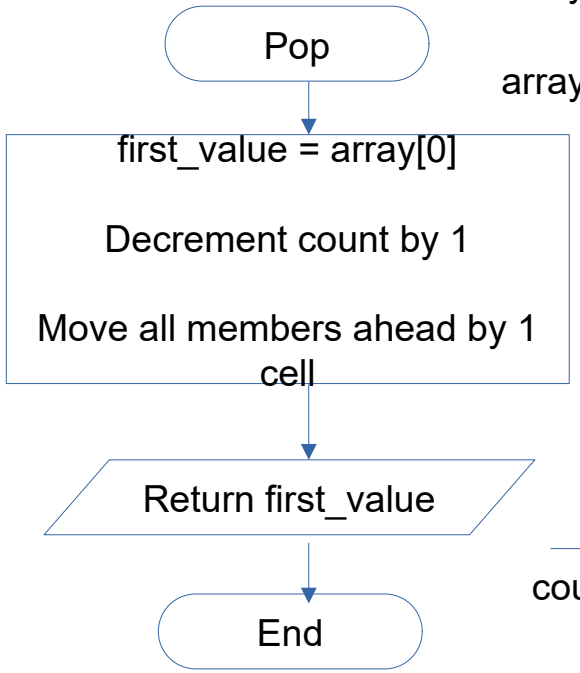


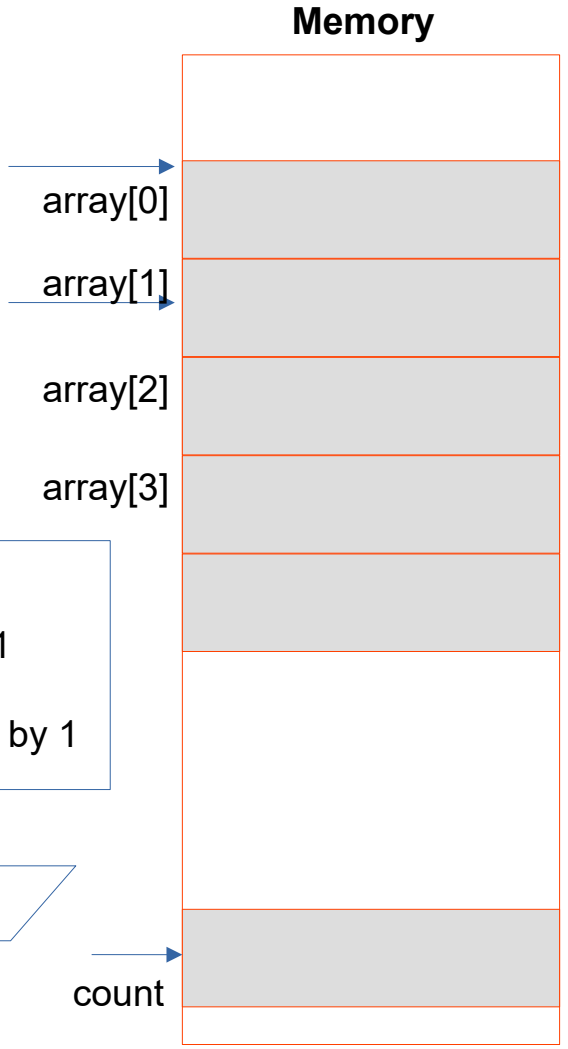
Queue Data Structure (FIFO)

- Queue is data structure that can hold multiple items
- Queue has a **front** and a **rear** pointer
- An Item can be added to the Queue using **push** method.
- An Item can be removed from the Queue using **pop** method
- The Queue has a FIFO (First In First Out) policy. The **pop** method removes the item from the from and shifts remaining members to front by one position.

```
struct Queue
{
 Int [5] array;
 int count = 0;
}

Queue q1;
q1.push(10);
q1.push(20);
```







Structures Using Key-Value Pair

Dictionary Map

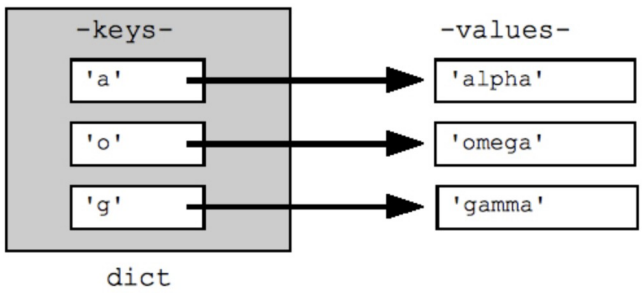
- It is similar to a list where each member has a key name and a value associated with it
- The keys are unique and duplicate keys are not allowed

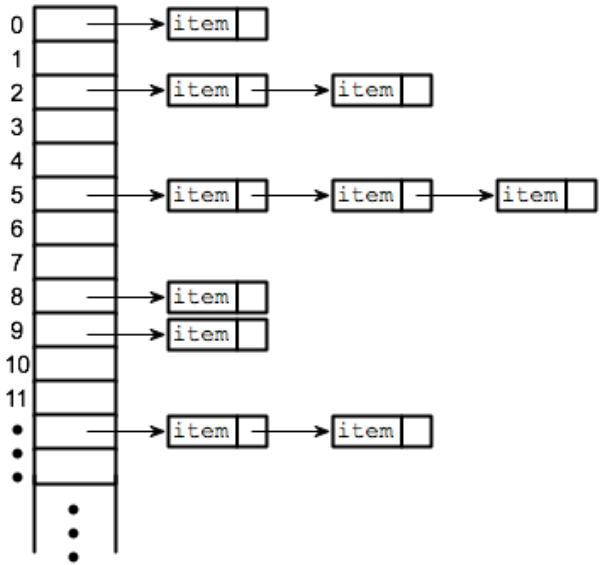
Applications :

- Fast data retrieval
- Caching

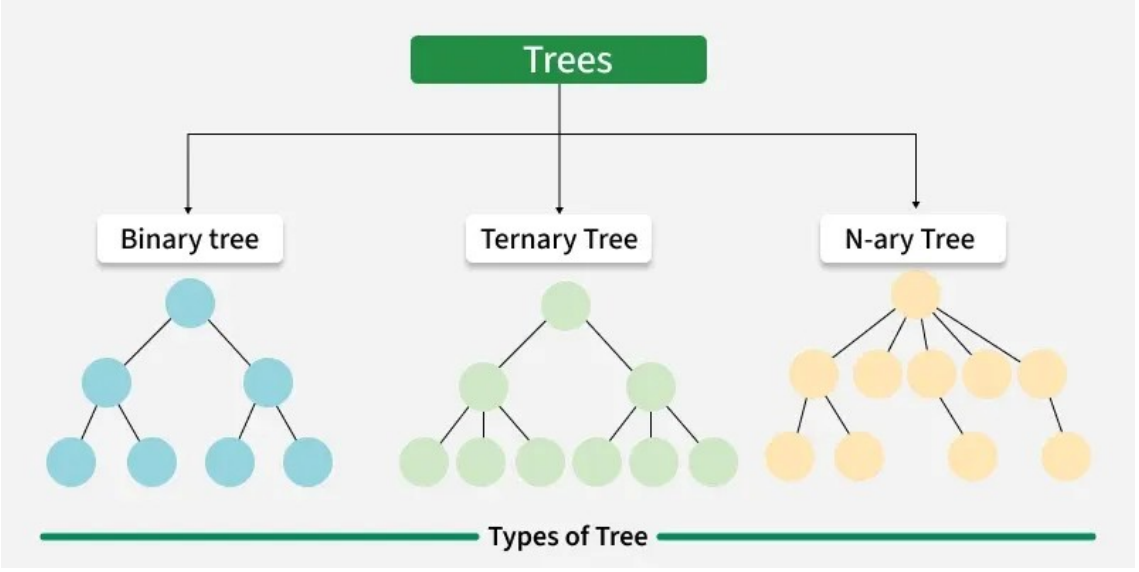
HashTable Map

- It is similar to dictionary except that the each keys maps to multiple values.
- The key is calculated by the hashtable and is called hash of the values.



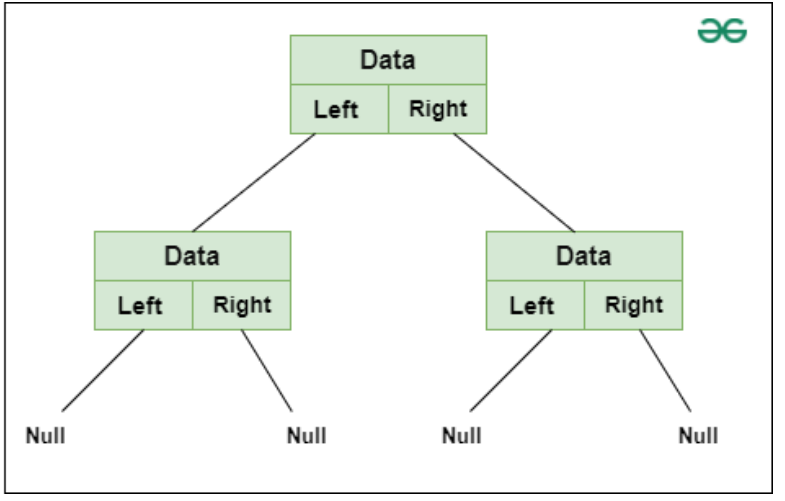


Tree Data Type

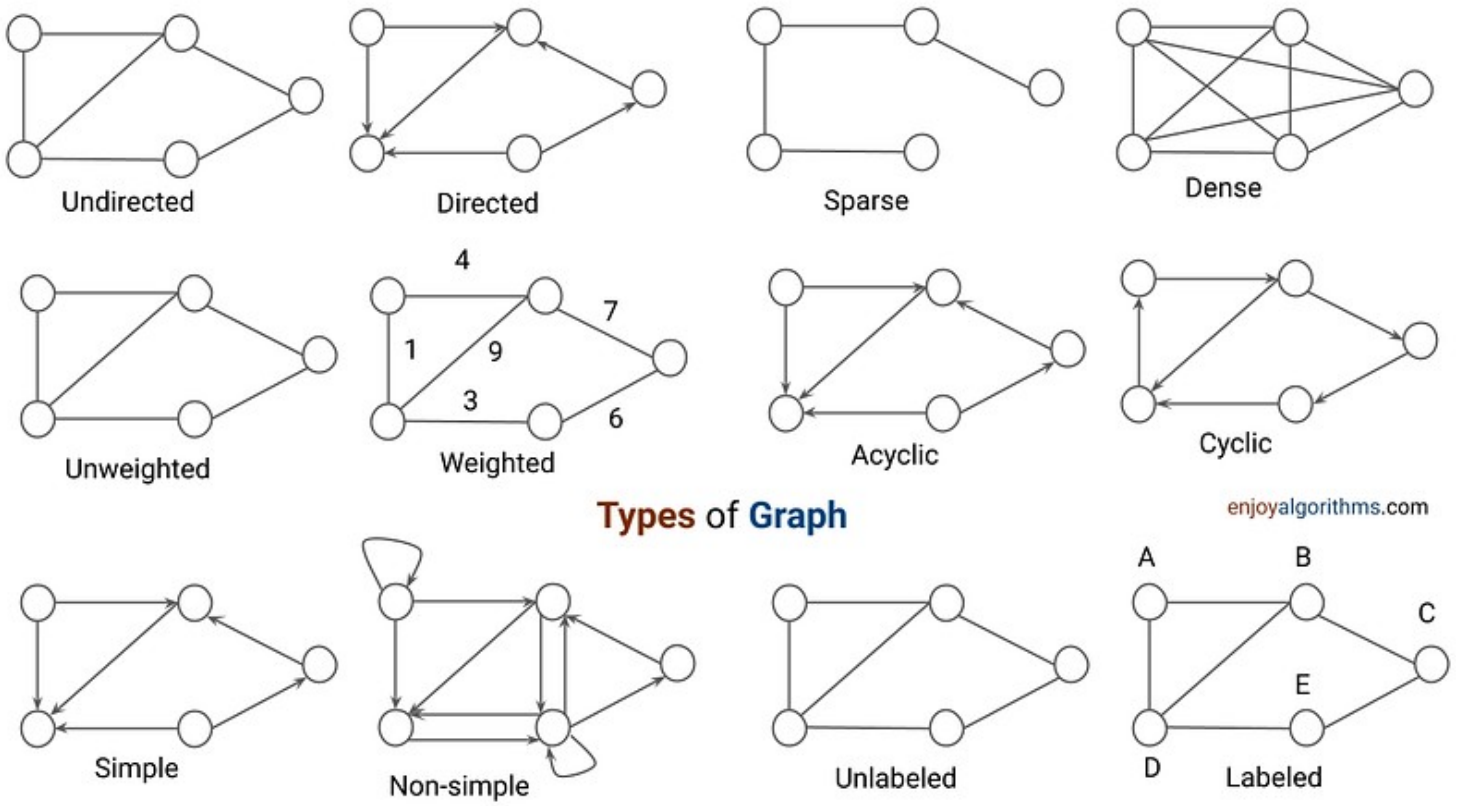


Applications :

- Heptarchical data
- Efficient Search and Sort algorithms



Graph Data Types



Applications :

- Computer networks
- Finding shortest path
- Navigation

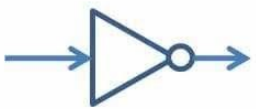
Logical Operations

Logical Operators

- Logical operations on binary variables, where variables can be either true (1) or false (0)

NOT

x	F
0	1
1	0



AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

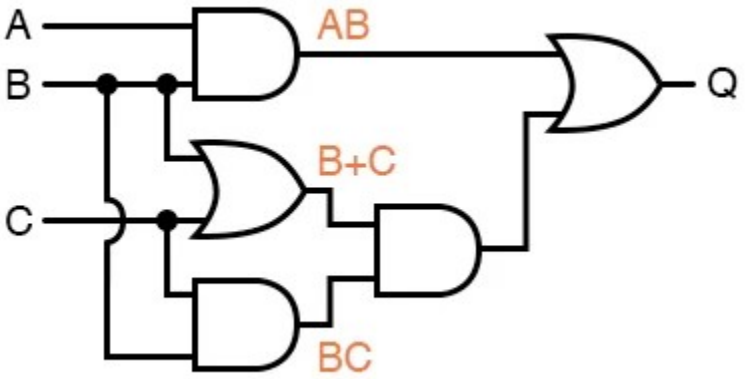
XOR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	0

Boolean Algebra (Exercise 1)

- Let A = 1, B=0, C=1

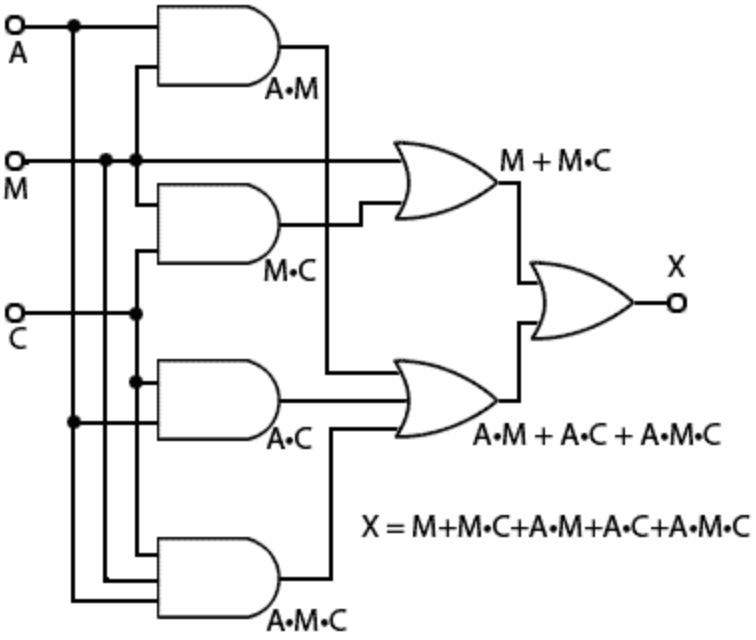
  (1) AB = A AND B = ?
  (2) BC = B AND C = ?
  (3) B+C = B OR C = ?
  (4) Q = AB OR ((B OR C) AND (B AND C))



Boolean Algebra (Exercise 2)

- Let A = 1, M=1, C=0

(1) M OR (M AND C) = ?
(2) (A AND M AND C) = ?
(3) (A AND M) OR (A AND C) OR (A AND M AND C) = ?
(4) X = ?



Algorithms

Algorithms

Searching Algorithms

- **Array Search**
  - Linear Search
  - Binary Search
  - Hashing Algorithm
- **Tree/Graph Search**
  - Breadth First Search
  - Depth First Search
  - Shortest Path Search

Sorting Algorithms

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort

String Algorithms

- String search
- String Reversal
- Regular Expressions

Machine Learning Algorithms

- Supervised (Regression, Classification)
- Unsupervised (Clustering)
- Reinforcement

Robotic Algorithms

- Path Planning
- Motion Planning
- SLAM

Load Balancing Algorithms

- Static
  - Round Robin
  - Weighted
  - Random
- Dynamic
  - Least Connections
  - URL Hashing
  - Geo-Locaiton Based

String Algorithms – String Reversal Example

Input Array = HELLO!

H	E	L	L	O	!

Length = 6
Start = 0
End = 5

PASS 1: Start = 0, End = 5, 0 < 5 (YES)

!	E	L	L	O	H

Start End

PASS 2: Start = 1, End = 4, 1 < 4 (YES)

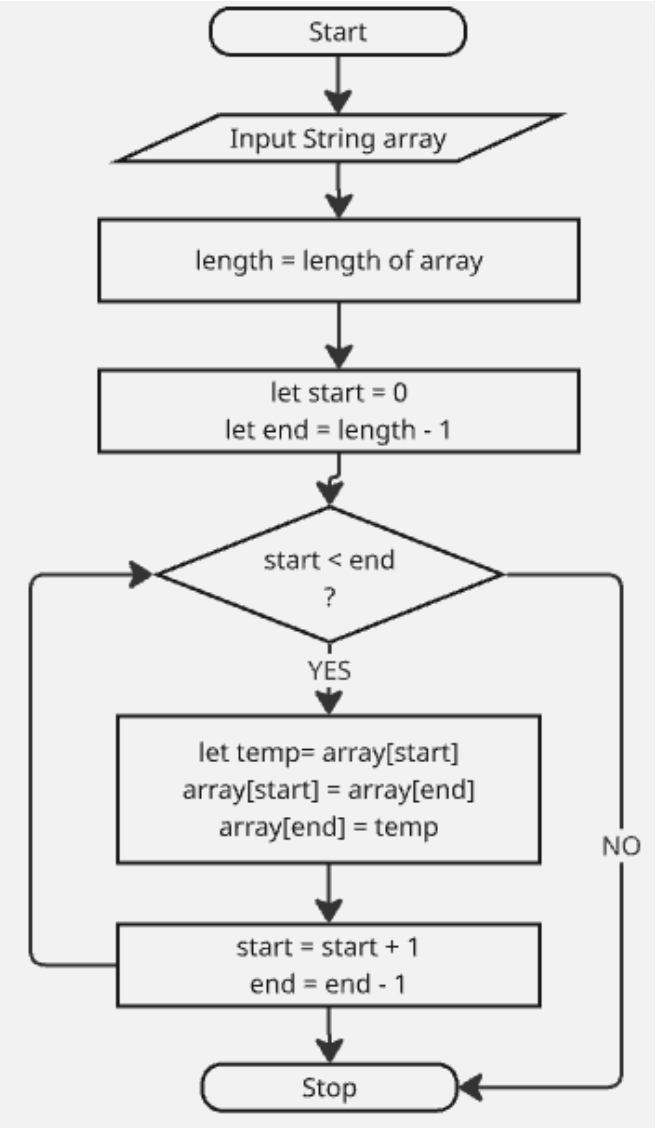
!	O	L	L	E	H

Start End

!	E	L	L	E	H

Start End

PASS 4: Start = 3, End = 2, 3 < 2 (NO)



Searching Algorithms – Linear Search (Single value)

Input Array = HELLO!

H	E	L	L	O	!

Length = 6
Search value = ‘L’
Index = 0

PASS 1: index= 0, 0 < 6 (YES)

H	E	L	L	O	!

index

No Match

PASS 2: index = 1, 1 < 6 (YES)

!	O	L	L	E	H

index

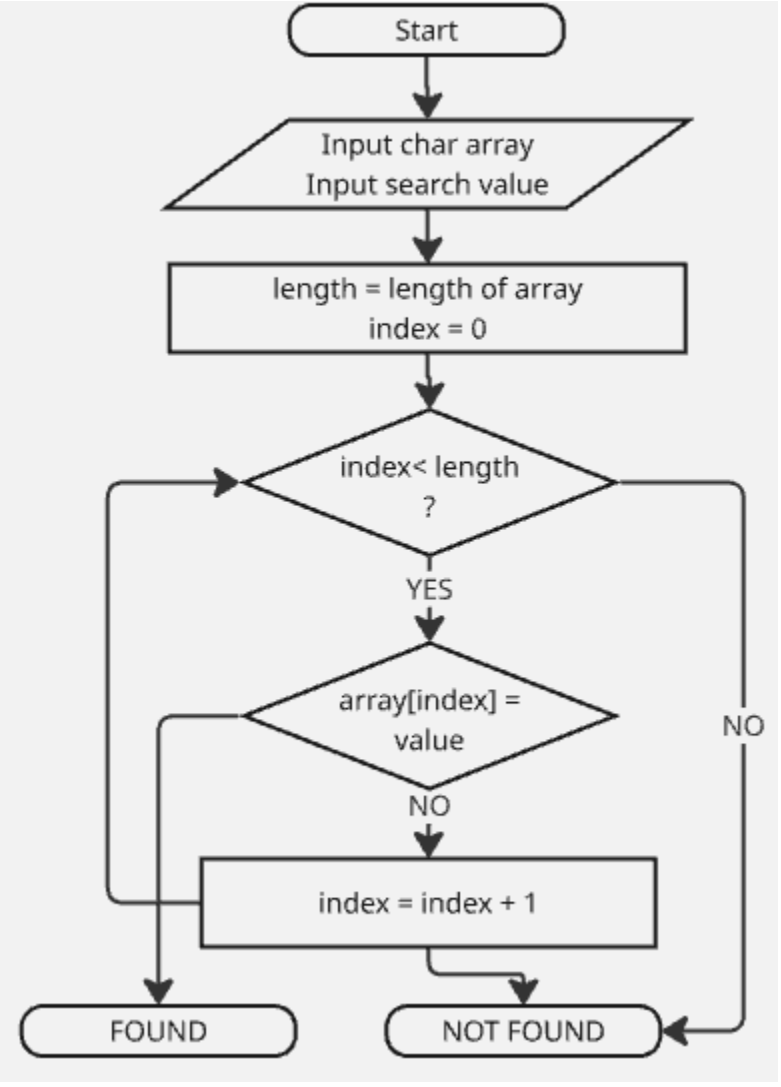
No Match

PASS 3: index = 2, 2 < 6 (YES)

!	E	L	L	E	H

index

MATCH



Searching Algorithms – Linear Search (sequence search)